Voucher Allocation Linear Programming Model Documentation

Overview

This document provides comprehensive documentation for the `LP.py` script, which implements a Linear Programming (LP) model to optimize voucher allocation across different tenant tiers. The model maximizes customer spending while respecting budget constraints and minimum allocation requirements.

Problem Statement

The model solves the following optimization problem:

- **Objective**: Maximize total customer dollar spending generated by voucher allocation
- **Constraints**: Stay within the company's ION loss budget while ensuring each tenant receives minimum vouchers
- **Decision**: Determine optimal number of vouchers to allocate to each tier

Model Parameters

Input Parameters

Parameter	Variable	Example Value	Description
Number of Tiers	`n`	`3`	Total number of voucher tiers available
Customer Spend	`c`	`[4, 5.7, 10]`	Dollar amount customers spend per voucher, by tier
ION Loss per Voucher	`y`	`[0.8, 2.8, 4.8]`	Company cost/loss per voucher issued, by tier
Total Budget	`B`	`420`	Maximum ION loss budget available
Tenants per Tier	`N`	`[10, 5, 2]`	Number of tenants subscribed to each tier
Minimum Vouchers	`Z`	`5`	Minimum vouchers each tenant must receive
Revenue Tiers	`R`	`[50, 125, 225, 450]`	Subscription revenue per tier

Calculated Parameters

- **Tenant Percentage** (`p`): Proportion of total tenants in each tier
  - Formula: `p[i] = N[i] / sum(N)`
  - Used to set tier capacity constraints

Decision Variables

- `x[i]`: Number of vouchers to allocate to tier `i`
  - Non-negative continuous variables
  - Can be modified to integer variables by adding `cat='Integer'`

Mathematical Formulation

Objective Function

Maximize: `∑(c[i] × x[i])` for all tiers i

This maximizes total expected customer spending across all tiers.

Constraints

1. **Budget Constraint**:
   - `∑(y[i] × x[i]) ≤ B`
   - Total ION loss cannot exceed available budget
2. **Tier Capacity Constraint**:
   - `x[i] ≤ p[i] × B` for each tier i
   - Limits allocation based on tier's proportion of tenants
3. **Minimum Allocation Constraint**:
   - `x[i] ≥ N[i] × Z` for each tier i
   - Ensures each tenant receives minimum required vouchers

Pre-Processing Functions

Revenue Calculation

```
def total_revenue(R, N):
```

- Calculates total subscription revenue from all tenants
- Used for informational purposes, not optimization
- Processes tiers in reverse order

Budget Validation

```
def calculate_min_budget(N, Z, y, B):
```

- Calculates minimum budget required: `∑(N[i] × Z × y[i])`
- Validates that provided budget B meets minimum requirements
- Raises error if budget is insufficient

Implementation Details

Model Setup

1. **Problem Definition**: Created as maximization problem using PuLP
2. **Variable Creation**: Decision variables with non-negative bounds
3. **Constraint Addition**: All constraints added to model object
4. **Solver Execution**: Default solver used to find optimal solution

Output Interpretation

The model provides three key outputs:

1. **Status**: Indicates solution quality
   - "Optimal": Best solution found
   - "Infeasible": No solution satisfies all constraints
   - "Unbounded": Objective can increase indefinitely
2. **Objective Value**: Maximum achievable customer spending
3. **Variable Values**: Optimal voucher allocation per tier
   - `x_0`, `x_1`, `x_2`, etc. represent vouchers for each tier

Usage Guidelines

Input Modification

- Adjust parameter values at the top of the script
- Ensure arrays (c, y, N) have consistent length matching n
- Verify budget B is realistic for the scenario

Model Extensions

- Add integer constraints for whole voucher numbers
- Include additional business constraints
- Modify objective to include other business metrics

Troubleshooting

- **Infeasible**: Increase budget B or reduce minimum requirements Z
- **Unbounded**: Add upper bound constraints on allocations
- **No solution**: Check parameter consistency and constraint feasibility

Example Scenario

With the default parameters:

- 3 voucher tiers with different spending and cost profiles
- 17 total tenants (10 + 5 + 2)
- $420 ION loss budget
- 5 minimum vouchers per tenant

The model determines the optimal allocation that maximizes customer spending while respecting all constraints.

Technical Requirements

- **Python Library**: PuLP for linear programming
- **Installation**: `pip install pulp`
- **Solver**: Uses default LP solver (typically CBC)

Customization Options

1. **Integer Variables**: Uncomment `cat='Integer'` for whole numbers
2. **Additional Constraints**: Add business-specific rules
3. **Multi-Objective**: Extend to consider multiple business goals
4. **Sensitivity Analysis**: Modify parameters to test different scenarios

This documentation enables users to understand, modify, and explain the voucher allocation optimization model effectively.